Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 6

Дисциплина МДК 03.02

Отчёт по тестированию программного обеспечения

Вариант № 2
АИС «Умный кампус»
Модуль Library System

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3
1.1 Основания и условия испытаний.....3
1.2 Результаты и дефекты.....4
1.3 BUG-001 Critical.....4
1.4 BUG-002 Critical.....5
1.5 BUG-003 Major.....5
1.6 BUG-004 Minor.....5
1.7 Выводы.....6
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....7
ПРИЛОЖЕНИЕ А Листинг исходного кода.....8
ПРИЛОЖЕНИЕ Б Матрица покрытия и реестр тестов.....14

1 ВЫПОЛНЕНИЕ РАБОТЫ

Я выполнил работу по варианту 2 Library System АИС «Умный кампус» на C# и .NET 8. Модуль принимает PDF/EPUB/ZIP в карантин; права Reader и Librarian разделены [3]. Оформление выполнено по положению института [2].

1.1 Основания и условия испытаний

Я составил LS-TP-01 по FR-01–FR-10. Методика согласована с ГОСТ 19.301-79, структура случаев, результатов и дефектов — с ISO/IEC/IEEE 29119-3:2021 [1] [5].

Использованы xUnit 2.9.3 и FluentAssertions 6.12.2. Модульные тесты проверяют FilePolicy и CodeQuality; интеграционные — WebApplicationFactory, настоящую SQLite и отдельный временный каталог каждого ApiHarness [6] [4].

Данные — собственные PDF/EPUB и ZIP. Условия: .NET 8 SDK, восстановленные пакеты, успешная сборка. Критерии: нет Failed/Skipped, статусы совпадают с ожидаемыми, отказ ZIP не оставляет файлов. Матрица покрытия и полный реестр случаев — в приложении Б.

Сведения представлены в таблице 1.

Таблица 1 — Основные тестовые случаи

Тип	Вход	Ожидаемый результат
Позитивный	Librarian + PDF / EPUB	201, Quarantined
Позитивный	ZIP с двумя книгами	201, две записи
Негативный	../escape.pdf внутри ZIP	400, нет записи файлов
Негативный	Reader / нет токена	403 / 401
Негативный	EXE или MZ под видом PDF	400
Граница	0 / 1 МиБ / 1 МиБ + 1	400 / 201 / 413
Граница	12 / 13 записей ZIP	Приём / отказ
Граница	4-я загрузка за минуту	429

Источник: составлено автором по результатам работы

1.2 Результаты и дефекты

Фактический прогон: всего 47 случаев xUnit, Passed — 47, Failed — 0, Skipped — 0. Теории развёрнуты в отдельные случаи: это не 47 разных методов. Исходный журнал — verification/library-tests.trx; C#-разбор — tests.json. Повторный запуск выполнен в обозревателе тестов Visual Studio.

Отдельно выполнены 12 HTTP-сценариев на запущенном Kestrel; все прошли. Они пересекаются с интеграционными тестами и не складываются с ними в число независимых проверок. Матрица приложения Б отражает требования, не процент покрытия строк. Точная граница multipart, аварийное восстановление и длительная многопользовательская нагрузка не измерялись.

Сведения представлены в таблице 2.

Таблица 2 — Реестр дефектов

ID	Критичность	Суть	Состояние
BUG-001	Critical	ZIP выходит за каталог	Исправлен; регрессия пройдена
BUG-002	Critical	Произвольные байты сохраняются как книга	Исправлен; регрессия пройдена
BUG-003	Major	Reader допускается к записи	Исправлен; регрессия пройдена
BUG-004	Minor	Одинаковая строка error для разных отказов	Исправлен; регрессия пройдена

Источник: составлено автором по результатам работы

1.3 BUG-001 Critical

Воспроизведение исходного дефекта: ../escape.pdf. Ожидаемый результат: Отказ до записи. Контроль исправления: ZipSlipLeavesNoFiles. Дефект выявлен статически в before, регрессионный тест выполнен на исправленном API.

1.4 BUG-002 Critical

Воспроизведение исходного дефекта: MZ под видом PDF. Ожидаемый результат: 400. Контроль исправления: SpoofedPdfRejected. Дефект выявлен статически в before, регрессионный тест выполнен на исправленном API.

1.5 BUG-003 Major

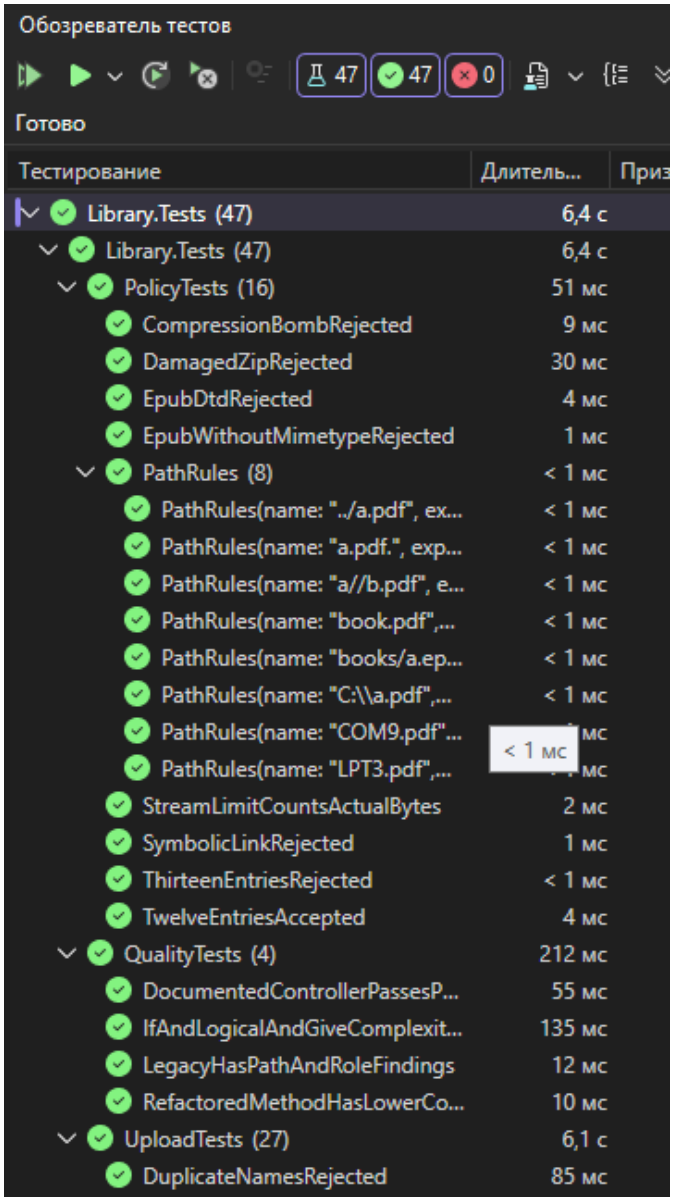
Воспроизведение исходного дефекта: Reader + корректный PDF. Ожидаемый результат: 403. Контроль исправления: ReaderCannotUpload. Дефект выявлен статически в before, регрессионный тест выполнен на исправленном API.

1.6 BUG-004 Minor

Воспроизведение исходного дефекта: Пустой файл и неверная роль. Ожидаемый результат: Различимые HTTP-статусы. Контроль исправления: EmptyFileRejected. Дефект выявлен статически в before, регрессионный тест выполнен на исправленном API.

BUG-004 имеет Minor: он затрагивал различимость сообщения об отказе, а не самостоятельный обход доступа. В рабочем решении используются различимые HTTP-статусы и контролируемая ошибка. Уязвимый обработчик не публиковался в сети ради демонстрации.

На рисунке 1 показан запуск в обозревателе тестов Visual Studio: 47 случаев, все 47 прошли, ошибок 0.



Источник: снимок прогона на Windows, предоставленный автором.

Рисунок 1 — Результат запуска xUnit-тестов в обозревателе тестов Visual Studio

1.7 Выводы

Я выполнил модульные, интеграционные и сквозные проверки. Все 47 случаев прошли, четыре исходных дефекта устранены и закреплены регрессией. Вывод ограничен указанными условиями и сценариями и допускает точное повторение по журналам.

Листинг — в приложении А. Код, журналы и запуск — в репозитории.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.301-79. ЕСПД. Программа и методика испытаний. Требования к содержанию и оформлению. — URL: https://protect.gost.ru/gost/details/b25e0645-091a-425b-8558-3e0c3ce37315 (дата обращения: 08.11.2026) [1].

2. Положение об оформлении письменных работ. Санкт-Петербургский институт экономики и управления. Методические материалы [2].

3. Практические работы МДК 03.02. Задания ПР 1–8 и эссе. Вариант 2 Library System. Материалы преподавателя [3].

4. FluentAssertions. Introduction. — URL: https://fluentassertions.com/introduction (дата обращения: 08.11.2026) [4].

5. ISO/IEC/IEEE 29119-3:2021. Software testing. Part 3. Test documentation. — URL: https://www.iso.org/standard/79429.html (дата обращения: 08.11.2026) [5].

6. xUnit.net. Getting started with xUnit.net v2 using the .NET SDK. — URL: https://xunit.net/docs/getting-started/v2/getting-started (дата обращения: 08.11.2026) [6].

ПРИЛОЖЕНИЕ А
Листинг исходного кода

tests/Library.Tests/ApiHarness.cs

```
using Library.Api;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
namespace Library.Tests;

public sealed class ApiHarness : IDisposable
{
   public string Root { get; } = Path.Combine(Path.GetTempPath(), "LibraryTests",
Guid.NewGuid().ToString("N"));
   public WebApplicationFactory<Program> Factory { get; }
   public HttpClient Client { get; }
   public ApiHarness(string environment = "Development")
   {
      Factory = new WebApplicationFactory<Program>().WithWebHostBuilder(builder => {
         builder.UseEnvironment(environment);
         builder.ConfigureAppConfiguration((_, cfg) => cfg.AddInMemoryCollection(new
Dictionary<string,string?> { ["StorageRoot"] = Root }));
      });
      Client = Factory.CreateClient();
   }
   public void Role(string role, string actor = "alice")
   {
      string token = Factory.Services.GetRequiredService<TokenRegistry>().Issue(role, actor);
      Client.DefaultRequestHeaders.Authorization = new("Bearer", token);
   }
   public async Task<HttpResponseMessage> Upload(byte[] bytes, string filename = "book.pdf", string title =
"Книга")
   {
      using var data = new MultipartFormDataContent();data.Add(new StringContent(title), "title");
      data.Add(new ByteArrayContent(bytes), "file", filename);
      return await Client.PostAsync("/api/books/upload", data);
   }
   public void Dispose()
   {
      Client.Dispose();Factory.Dispose();
      Microsoft.Data.Sqlite.SqliteConnection.ClearAllPools();
      if (Directory.Exists(Root)) Directory.Delete(Root, true);
   }
}
```

tests/Library.Tests/UploadTests.cs

```
using System.Net;
using System.Text;
using System.Text.Json;
using FluentAssertions;
using Library.Core;
using Xunit;
namespace Library.Tests;

public sealed class UploadTests
```

```
{
    [Fact]
    public async Task InternalFailureDoesNotLeakPathOrStack()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        string target = Path.Combine(h.Root, "quarantine");
        Directory.Delete(target);File.WriteAllText(target, "blocked");
        var response = await h.Upload(Samples.Pdf());
        response.StatusCode.Should().Be(HttpStatusCode.InternalServerError);
        string text = await response.Content.ReadAsStringAsync();
        text.Should().Contain("server_error").And.NotContain(h.Root).And.NotContain("StackTrace");
    }
    [Fact]
    public async Task LibrarianUploadsPdfToQuarantine()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        var r = await h.Upload(Samples.Pdf());r.StatusCode.Should().Be(HttpStatusCode.Created);
        string result = await r.Content.ReadAsStringAsync();result.Should().Contain("Quarantined");
        var files = Directory.GetFiles(Path.Combine(h.Root, "quarantine"));files.Should().HaveCount(1);
        Path.GetFileName(files[0]).Should().NotBe("book.pdf");
    }
    [Fact]
    public async Task LibrarianUploadsEpub()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Epub(), "book.epub")).StatusCode.Should().Be(HttpStatusCode.Created);
    }
    [Fact]
    public async Task SafeZipCreatesTwoBooks()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Zip(("a.pdf", Samples.Pdf()), ("b.epub", Samples.Epub())), "books.zip"))
            .StatusCode.Should().Be(HttpStatusCode.Created);
        Directory.GetFiles(Path.Combine(h.Root, "quarantine")).Should().HaveCount(2);
    }
    [Fact]
    public async Task ReaderCannotUpload()
    {
        using var h = new ApiHarness();h.Role("Reader");
        (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.Forbidden);
    }
    [Fact]
    public async Task MissingTokenRejected()
    {
        using var h = new ApiHarness();
        (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.Unauthorized);
    }
    [Fact]
    public async Task InvalidTokenRejected()
    {
        using var h = new ApiHarness();h.Client.DefaultRequestHeaders.Authorization = new("Bearer",
"invalid");
        (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.Unauthorized);
    }
    [Theory]
    [InlineData("../escape.pdf")]
    [InlineData("..\\escape.pdf")]
    [InlineData("/absolute.pdf")]
    [InlineData("C:\\escape.pdf")]
```

```
    [InlineData("a/../../escape.pdf")]
    public async Task ZipSlipLeavesNoFiles(string path)
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Zip((path, Samples.Pdf())),
"bad.zip")).StatusCode.Should().Be(HttpStatusCode.BadRequest);
        Directory.GetFiles(Path.Combine(h.Root, "quarantine")).Should().BeEmpty();
        File.Exists(Path.Combine(h.Root, "escape.pdf")).Should().BeFalse();
    }
    [Fact]
    public async Task SpoofedPdfRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload("MZ executable"u8.ToArray())).StatusCode.Should().Be(HttpStatusCode.BadRequest);
    }
    [Fact]
    public async Task ExecutableRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload("MZ"u8.ToArray(), "bad.exe")).StatusCode.Should().Be(HttpStatusCode.BadRequest);
    }
    [Fact]
    public async Task EmptyFileRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload([])).StatusCode.Should().Be(HttpStatusCode.BadRequest);
    }
    [Fact]
    public async Task ExactOneMebibyteAccepted()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        byte[] bytes = new byte[FilePolicy.BookLimit];Samples.Pdf().CopyTo(bytes,0);"%
%EOF"u8.CopyTo(bytes.AsSpan(bytes.Length-5));
        (await h.Upload(bytes)).StatusCode.Should().Be(HttpStatusCode.Created);
    }
    [Fact]
    public async Task OneByteOverBookLimitRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(new
byte[FilePolicy.BookLimit+1])).StatusCode.Should().Be(HttpStatusCode.RequestEntityTooLarge);
    }
    [Fact]
    public async Task DuplicateNamesRejected()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Zip(("a.pdf", Samples.Pdf()), ("A.pdf", Samples.Pdf())), "bad.zip"))
            .StatusCode.Should().Be(HttpStatusCode.BadRequest);
    }
    [Fact]
    public async Task MixedZipSavesNothing()
    {
        using var h = new ApiHarness();h.Role("Librarian");
        (await h.Upload(Samples.Zip(("a.pdf", Samples.Pdf()), ("bad.exe", "MZ"u8.ToArray())), "bad.zip"))
            .StatusCode.Should().Be(HttpStatusCode.BadRequest);
        Directory.GetFiles(Path.Combine(h.Root, "quarantine")).Should().BeEmpty();
    }
    [Fact]
    public async Task FourthUploadRateLimited()
```

```
  {
    using var h = new ApiHarness();h.Role("Librarian");
    for (int i=0;i<3;i++) (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.Created);
    (await h.Upload(Samples.Pdf())).StatusCode.Should().Be(HttpStatusCode.TooManyRequests);
  }
  [Fact]
  public async Task OtherLibrarianCannotSeeQuarantine()
  {
    using var h = new ApiHarness();h.Role("Librarian", "alice");await h.Upload(Samples.Pdf(),title:"Private
title");
    h.Role("Librarian", "bob");
    (await h.Client.GetStringAsync("/api/books")).Should().NotContain("Private title");
  }
  [Fact]
  public async Task QuarantineCannotBeDownloaded()
  {
    using var h = new ApiHarness();h.Role("Librarian");var r=await h.Upload(Samples.Pdf());
    string id=JsonDocument.Parse(await
r.Content.ReadAsStringAsync()).RootElement[0].GetProperty("id").GetString()!;
    (await h.Client.GetAsync("/api/books/"+id+"/file")).StatusCode.Should().Be(HttpStatusCode.NotFound);
  }
  [Fact]
  public async Task ReaderDownloadsPublishedSample()
  {
    using var h = new ApiHarness();h.Role("Reader");
    var r=await h.Client.GetAsync("/api/books/11111111-1111-4111-8111-111111111111/file");
    r.StatusCode.Should().Be(HttpStatusCode.OK);r.Headers.GetValues("X-Content-Type-
Options").Should().Contain("nosniff");
  }
  [Fact]
  public async Task SqlLikeQueryRemainsData()
  {
    using var h = new ApiHarness();h.Role("Reader");
    var r=await h.Client.GetAsync("/api/books?query="+Uri.EscapeDataString("' OR 1=1 --"));
    r.StatusCode.Should().Be(HttpStatusCode.OK);(await r.Content.ReadAsStringAsync()).Should().Be("[]");
  }
  [Fact]
  public async Task ProductionHidesLabSession()
  {
    using var h = new ApiHarness("Production");
    (await h.Client.GetAsync("/lab/session/Librarian")).StatusCode.Should().Be(HttpStatusCode.NotFound);
  }
  [Fact]
  public async Task RejectedUploadIsAuditedWithoutToken()
  {
    using var h = new ApiHarness();h.Role("Reader");await h.Upload(Samples.Pdf());
    string log=File.ReadAllText(Path.Combine(h.Root,"audit.jsonl"));
    log.Should().Contain("403").And.Contain("alice").And.NotContain("Bearer");
  }
  [Fact]
  public async Task LongTitleRejected()
  {
    using var h = new ApiHarness();h.Role("Librarian");
    (await h.Upload(Samples.Pdf(),title:new
string('A',101))).StatusCode.Should().Be(HttpStatusCode.BadRequest);
  }
}
```

tests/Library.Tests/PolicyTests.cs

```
using System.IO.Compression;
```

```
using System.Text;
using FluentAssertions;
using Library.Core;
using Xunit;
namespace Library.Tests;
public sealed class PolicyTests
{
   [Theory]
   [InlineData("book.pdf",true)]
   [InlineData("books/a.epub",true)]
   [InlineData("../a.pdf",false)]
   [InlineData("C:\\a.pdf",false)]
   [InlineData("COM9.pdf",false)]
   [InlineData("a.pdf.",false)]
   [InlineData("a//b.pdf",false)]
   [InlineData("LPT3.pdf",false)]
   public void PathRules(string name,bool expected) => FilePolicy.IsSafeName(name).Should().Be(expected);
   [Fact]
   public void ThirteenEntriesRejected()
   {
      byte[] zip=Samples.Zip(Enumerable.Range(0,13).Select(i=>($"{i}.pdf",Samples.Pdf())).ToArray());
      Action act=()=>FilePolicy.Validate("bad.zip",zip);act.Should().Throw<RejectedFile>();
   }
   [Fact]
   public void TwelveEntriesAccepted()
   {
      byte[] zip=Samples.Zip(Enumerable.Range(0,12).Select(i=>($"{i}.pdf",Samples.Pdf())).ToArray());
      FilePolicy.Validate("good.zip",zip).Should().HaveCount(12);
   }
   [Fact]
   public void DamagedZipRejected()
   {
      Action
act=()=>FilePolicy.Validate("bad.zip","PKbroken"u8.ToArray());act.Should().Throw<RejectedFile>();
   }
   [Fact]
   public void StreamLimitCountsActualBytes()
   {
      Action act=()=>FilePolicy.ReadLimited(new MemoryStream(new byte[11]),10);
      act.Should().Throw<RejectedFile>().Which.Status.Should().Be(413);
   }
   [Fact]
   public void EpubWithoutMimetypeRejected()
   {
      Action act=()=>FilePolicy.Validate("bad.epub",Samples.Zip(("a.xhtml","text"u8.ToArray())));
      act.Should().Throw<RejectedFile>();
   }
   [Fact]
   public void EpubDtdRejected()
   {
      byte[] zip=Samples.Zip(("mimetype","application/epub+zip"u8.ToArray()),
         ("META-INF/container.xml",Encoding.UTF8.GetBytes("<!DOCTYPE x [<!ENTITY a SYSTEM
'file:///never'>]><container>&a;</container>")));
      Action act=()=>FilePolicy.Validate("bad.epub",zip);act.Should().Throw<RejectedFile>();
   }
   [Fact]
   public void SymbolicLinkRejected()
   {
      using var data=new MemoryStream();
```

```
        using(var zip=new ZipArchive(data,ZipArchiveMode.Create,true))
        {var e=zip.CreateEntry("link.pdf");e.ExternalAttributes=unchecked((int)0xA1FF0000);using var
s=e.Open();s.Write(Samples.Pdf());}
        Action act=()=>FilePolicy.Validate("link.zip",data.ToArray());act.Should().Throw<RejectedFile>();
    }
    [Fact]
    public void CompressionBombRejected()
    {
        using var data=new MemoryStream();
        using(var zip=new ZipArchive(data,ZipArchiveMode.Create,true))
        {using var s=zip.CreateEntry("big.pdf",CompressionLevel.SmallestSize).Open();s.Write(new
byte[500000]);}
        Action act=()=>FilePolicy.Validate("bomb.zip",data.ToArray());act.Should().Throw<RejectedFile>();
    }
}
```

tests/Library.Tests/QualityTests.cs

```
using FluentAssertions;
using Library.Tools;
using Xunit;
namespace Library.Tests;
public sealed class QualityTests
{
    private static string Source(string relative)
    {
        var directory = new DirectoryInfo(Directory.GetCurrentDirectory());
        while (directory is not null && !Directory.Exists(Path.Combine(directory.FullName, "artifacts")))
directory = directory.Parent;
        return File.ReadAllText(Path.Combine(directory!.FullName, relative));
    }
    [Fact]
    public void RefactoredMethodHasLowerComplexityAndHigherMi()
    {
        var before = CodeQuality.Measure(Source("before/LegacyUploadController.cs"),
"LegacyUploadController").Methods[0];
        var after = CodeQuality.Measure(Source("src/Library.Api/UploadController.cs"),
"UploadController").Methods[0];
        after.Complexity.Should().BeLessThan(before.Complexity);
        after.Mi.Should().BeGreaterThan(before.Mi);
        string shared = "class C(int value) { public int F() => value; public int G() => value; }";
        CodeQuality.Measure(shared, "C").Lcom1.Should().Be(0);
    }
    [Fact]
    public void DocumentedControllerPassesProjectAudit() =>
        CodeQuality.Audit(Source("src/Library.Api/UploadController.cs")).Should().BeEmpty();
    [Fact]
    public void LegacyHasPathAndRoleFindings()
    {
        var findings = CodeQuality.Audit(Source("before/LegacyUploadController.cs"));
        findings.Select(x => x.Rule).Should().Contain(new[] { "AUD-04", "AUD-05" });
    }
    [Fact]
    public void IfAndLogicalAndGiveComplexityThree()
    {
        string source = "class C { public int F(bool a,bool b) { if(a && b) return 1; return 0; } }";
        CodeQuality.Measure(source, "C").Methods[0].Complexity.Should().Be(3);
    }
}
```

ПРИЛОЖЕНИЕ Б
Матрица покрытия и реестр тестов

Сведения представлены в таблице Б.1.

Таблица Б.1 — Покрытие требований

Требование	Критерий	Имена тестов
FR-01	Librarian загружает PDF, EPUB и ZIP; 201, Quarantined	LibrarianUploadsPdfToQuarantine; LibrarianUploadsEpub; SafeZipCreatesTwoBooks
FR-02	Reader получает 403; нет / неверный токен — 401	ReaderCannotUpload; MissingTokenRejected; InvalidTokenRejected
FR-03	Небезопасный архив не оставляет файлов	ZipSlipLeavesNoFiles; MixedZipSavesNothing; SymbolicLinkRejected
FR-04	Проверка формата и структуры независимо от MIME	SpoofedPdfRejected; ExecutableRejected; EpubWithoutMimetypeRejected; EpubDtdRejected; DamagedZipRejected
FR-05	Размеры, количество записей и защита от распаковки	EmptyFileRejected; ExactOneMebibyteAccepted; OneByteOverBookLimitRejected; TwelveEntriesAccepted; ThirteenEntriesRejected; CompressionBombRejected; StreamLimitCountsActualBytes

Источник: составлено автором по результатам работы

Продолжение таблицы Б.1

Требование	Критерий	Имена тестов
FR-06	3 загрузки в минуту на пользователя, затем 429	FourthUploadRateLimited
FR-07	Серверное имя и параметризация поиска EF Core	LibrarianUploadsPdfToQuarantine; SqlLikeQueryRemainsData
FR-08	Каталог скрывает чужой карантин; выдача только Published	OtherLibrarianCannotSeeQuarantine; QuarantineCannotBeDownloaded; ReaderDownloadsPublishedSample
FR-09	Отказы заносятся в журнал без токена	RejectedUploadIsAuditedWithoutToken
FR-10	Нет путей и stack trace в ответе, нет lab в Production	InternalFailureDoesNotLeakPathOrStack; ProductionHidesLabSession

Источник: составлено автором по результатам работы

Сведения представлены в таблице Б.2.

Таблица Б.2 — Все выполненные случаи xUnit

№	Имя случая	Итог
1	PolicyTests.CompressionBombRejected	Passed
2	PolicyTests.DamagedZipRejected	Passed
3	PolicyTests.EpubDtdRejected	Passed
4	PolicyTests.EpubWithoutMimetypeRejected	Passed
5	PolicyTests.PathRules(name: "../a.pdf", expected: False)	Passed
6	PolicyTests.PathRules(name: "a.pdf.", expected: False)	Passed
7	PolicyTests.PathRules(name: "a//b.pdf", expected: False)	Passed
8	PolicyTests.PathRules(name: "book.pdf", expected: True)	Passed
9	PolicyTests.PathRules(name: "books/a.epub", expected: True)	Passed
10	PolicyTests.PathRules(name: "C:\\a.pdf", expected: False)	Passed
11	PolicyTests.PathRules(name: "COM9.pdf", expected: False)	Passed
12	PolicyTests.PathRules(name: "LPT3.pdf", expected: False)	Passed
13	PolicyTests.StreamLimitCountsActualBytes	Passed
14	PolicyTests.SymbolicLinkRejected	Passed
15	PolicyTests.ThirteenEntriesRejected	Passed

Источник: составлено автором по результатам работы

Продолжение таблицы Б.2

№	Имя случая	Итог
16	PolicyTests.TwelveEntriesAccepted	Passed
17	QualityTests.DocumentedControllerPassesProjectAudit	Passed
18	QualityTests.IfAndLogicalAndGiveComplexityThree	Passed
19	QualityTests.LegacyHasPathAndRoleFindings	Passed
20	QualityTests.RefactoredMethodHasLowerComplexityAndHigherMi	Passed
21	UploadTests.DuplicateNamesRejected	Passed
22	UploadTests.EmptyFileRejected	Passed
23	UploadTests.ExactOneMebibyteAccepted	Passed
24	UploadTests.ExecutableRejected	Passed
25	UploadTests.FourthUploadRateLimited	Passed
26	UploadTests.InternalFailureDoesNotLeakPathOrStack	Passed
27	UploadTests.InvalidTokenRejected	Passed
28	UploadTests.LibrarianUploadsEpub	Passed
29	UploadTests.LibrarianUploadsPdfToQuarantine	Passed
30	UploadTests.LongTitleRejected	Passed

Источник: составлено автором по результатам работы

Продолжение таблицы Б.2

№	Имя случая	Итог
31	UploadTests.MissingTokenRejected	Passed
32	UploadTests.MixedZipSavesNothing	Passed
33	UploadTests.OneByteOverBookLimitRejected	Passed
34	UploadTests.OtherLibrarianCannotSeeQuarantine	Passed
35	UploadTests.ProductionHidesLabSession	Passed
36	UploadTests.QuarantineCannotBeDownloaded	Passed
37	UploadTests.ReaderCannotUpload	Passed
38	UploadTests.ReaderDownloadsPublishedSample	Passed
39	UploadTests.RejectedUploadIsAuditedWithoutToken	Passed
40	UploadTests.SafeZipCreatesTwoBooks	Passed
41	UploadTests.SpoofedPdfRejected	Passed
42	UploadTests.SqlLikeQueryRemainsData	Passed
43	UploadTests.ZipSlipLeavesNoFiles(path: "../escape.pdf")	Passed
44	UploadTests.ZipSlipLeavesNoFiles(path: "..\\escape.pdf")	Passed
45	UploadTests.ZipSlipLeavesNoFiles(path: "/absolute.pdf")	Passed

Источник: составлено автором по результатам работы

Продолжение таблицы Б.2

№	Имя случая	Итог
46	UploadTests.ZipSlipLeavesNoFiles(path: "a/../../escape.pdf")	Passed
47	UploadTests.ZipSlipLeavesNoFiles(path: "C:\\escape.pdf")	Passed

Источник: составлено автором по результатам работы